

Introduction :

Motion Detection System :

A motion detection system is a technology designed to detect movement within a specific area, widely used in security, automation, and interactive applications. It typically involves sensors (like cameras or infrared detectors) to sense motion, signal processing to analyze changes, and algorithms to distinguish between actual movement and background noise. Upon detecting motion, the system can trigger actions such as sounding alarms, capturing video, or activating lights. Motion detection systems enhance security, conserve energy, and improve user interaction, making them essential in both residential and commercial environments.

Implemented motion detection on a video to identify motion within the footage. The process was carried out using Python and essential libraries such as OpenCV and Mediapipe, each contributing significantly to enable computer vision in recognizing alterations in static pixels.

Brief regarding the libraries used :

OpenCV (Open Source Computer Vision Library)

OpenCV stands as a software library catering to computer vision and machine learning applications, offered free of charge. It serves a wide array of functions within the realms of computer vision and machine learning. The following offers a concise insight into its primary features and applications:

- 1. Image and Video Processing:** OpenCV excels in handling image and video processing tasks, including filtering, transformation, color space conversion, and histogram analysis.
- 2. Object Detection:** The library contains algorithms for detecting objects in images and videos, such as face detection, pedestrian detection, and vehicle detection.
- 3. Machine Learning:** OpenCV provides a comprehensive set of tools for training and deploying machine learning models. It supports various supervised and unsupervised learning algorithms like SVM, k-means clustering, and neural networks.
- 4. Feature Detection and Description:** OpenCV offers methods for detecting and describing features in images, such as corners, edges, and keypoints, essential for tasks like image stitching and object recognition.
- 5. 3D Vision:** OpenCV supports 3D vision tasks, including stereo vision, 3D reconstruction, and structure from motion.
- 6. Real-Time Applications:** Due to its optimized performance, OpenCV is ideal for real-time applications, facilitating the creation of real-time image and video processing systems.

MediaPipe :

An open-source framework created by Google, enables the construction of customizable machine learning pipelines that work across different platforms. It streamlines the creation of real-time computer vision and machine learning applications by offering a set of reusable components and comprehensive solutions. Here's a summary of its prominent features and applications:

- **Modular Design:** MediaPipe features a modular structure, allowing developers to build pipelines by combining multiple "calculators" that execute specific tasks like data input, processing, and output.
- **Cross-Platform Compatibility:** MediaPipe is compatible with various platforms, including mobile (Android, iOS), web (via WebAssembly), desktop (Windows, macOS, Linux), and embedded systems, facilitating the development of applications that work across platforms.
- **Pre-Built Solutions:** It includes pre-built solutions for common computer vision tasks, such as hand and pose tracking, face detection and mesh generation, object detection, and hair segmentation.
- **Real-Time Efficiency:** MediaPipe is optimized for real-time performance, making it ideal for applications like augmented reality, interactive media, and live video analysis that require rapid processing.
- **Customization:** Developers can tailor and expand the existing solutions by integrating their own algorithms or modifying existing components to meet specific requirements.
- **Seamless Integration:** MediaPipe is designed for easy integration with other machine learning frameworks like TensorFlow, allowing the smooth incorporation of complex models into MediaPipe pipelines.

Source Code :

```
import cv2
import mediapipe as mp
import time

mpDraw = mp.solutions.drawing_utils
mpPose = mp.solutions.pose
pose = mpPose.Pose()

cap = cv2.VideoCapture("workout.mp4")
pTime = 0

while True:

    success, img = cap.read()

    height, width, _ = img.shape
    max_width = 1000
    max_height = 800
    scale = min(max_width / width, max_height / height)
    img = cv2.resize(img, None, fx=scale, fy=scale)

    imgRGB = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
    result = pose.process(imgRGB)
    if result.pose_landmarks:
        mpDraw.draw_landmarks(
            img,
            result.pose_landmarks,
            mpPose.POSE_CONNECTIONS,
            landmark_drawing_spec=mpDraw.DrawingSpec(
                color=(0, 255, 0), thickness=2, circle_radius=2
            ),
            connection_drawing_spec=mpDraw.DrawingSpec(color=(0, 255, 0), thickness=2),
        )

    cTime = time.time()
    fps = 1 / (cTime - pTime)
    pTime = cTime
    cv2.putText(
        img, f"FPS : {int(fps)}", (20, 70), cv2.FONT_HERSHEY_PLAIN, 3, (0, 255, 0), 2
    )

    cv2.imshow("Image", img)
    if cv2.waitKey(10) & 0xFF == ord("q"):
        break
```

Code Breakdown:

Importing Libraries :

```
import cv2
import mediapipe as mp
import time
```

1. **CV2** : OpenCV library used for accessing computer vision tasks
OpenCV (Open Source Computer Vision) is a powerful library that provides a wide range of tools and functionalities for computer vision tasks.
2. **MediaPipe** : Libraries such as OpenPose play a vital role in computer vision by accurately identifying human poses using deep learning algorithms. OpenPose is a real-time multi-person system that detects body, hand, facial, and foot keypoints with precision, allowing analysis of human movement for sports, healthcare, and augmented reality.
3. **Time** : Standard Python Library used for time related functions. Here the library has been imported in order to calculate the **Frames Per Second (FPS)**.

Initializing Mediapipe Pose :

```
mpDraw = mp.solutions.drawing_utils  
mpPose = mp.solutions.pose  
pose = mpPose.Pose()
```

1. **mpDraw** : Utility Functions for drawing landmarks.
2. **mpPose** : Reference to Mediapipe's pose Solution.
3. **Pose** : An instance of the Pose class for porcessing images and detection pose landmarks.

Capturing Video :

```
cap = cv2.VideoCapture("workout.mp4")  
pTime = 0
```

1. **cap** : Captuire video from the file 'workout.mp4' using OpenCV.
2. **pTime** : Variable to store the previous time for FPS calculation.

Main Loop to Process Video Frames :

```
while True:  
    success, img = cap.read()
```

1. **success, img = cap.read()** : Reads a frame from the video.
`success` is a boolean indicating if the frame was read successfully.
`img` is the frame itself.

Resize Frame to Fit Maximum Dimensions :

```
height, width, _ = img.shape
max_width = 1000
max_height = 800
scale = min(max_width / width, max_height / height)
img = cv2.resize(img, None, fx=scale, fy=scale)
```

- Gets the original dimensions of the frame.
- Defines the maximum width and height for resizing.
- Calculates the scaling factor to fit frame within the maximum dimensions while maintaining aspect ratio.
- Resize the frame using the calculated scaling factor.

Convert Frame to RGB and Process Pose :

```
imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
result = pose.process(imgRGB)
```

1. **cv2.COLOR_BGR2RGB** : Converts the image from BGR to RGB (required by Mediapipe).
2. **pose.process(imgRGB)** : Processes the frame to detect pose landmarks.

Draw Pose Landmarks :

```
if result.pose_landmarks:
    mpDraw.draw_landmarks(
        img,
        result.pose_landmarks,
        mpPose.POSE_CONNECTIONS,
        landmark_drawing_spec=mpDraw.DrawingSpec(
            color=(0, 255, 0), thickness=2, circle_radius=2
        ),
        connection_drawing_spec=mpDraw.DrawingSpec(color=(0, 255, 0), thickness=2),
    )
```

- Checks if the pose landmarks are detected.
- **mpDraw.draw_landmarks()** : Draws the detected pose landmarks and connections on the frame. The use of “mpDraw.DrawingSpec()” is used to setup the specs for the drawn pose landmarks.

Calculate and Display FPS :

```
cTime = time.time()
fps = 1 / (cTime - pTime)
pTime = cTime
cv2.putText(
    img, f"FPS : {int(fps)}", (20, 70), cv2.FONT_HERSHEY_PLAIN, 3, (0, 255, 0), 2
)
```

1. **cTime = time.time()** : Gets the Current time.
2. **fps = 1/ (cTime-pTime)** : Calculates the FPS as the inverse of the time difference between the current and previous frames.
3. **pTime = cTime** : Updates `ptime` to the current time for the next frame.
4. **cv2.putText()** : Displays the FPS on the frame.

Display the Frame and Exit on Key Press :

```
cv2.imshow("Image", img)
if cv2.waitKey(10) & 0xFF == ord("q"):
    break
```

1. **cv2.imshow()** : Displays the Processed frame in a window.
2. **cv2.waitKey() == ord("q")** : Waits for 10 milliseconds for a key press and breaks the loop if the 'q' key is pressed .

Scope :

Multi-Person Pose Detection:

- **Objective:** Refine the script to simultaneously detect and track poses of multiple individuals.
- **Overview:** Integrate advanced algorithms or models to enable multi-person pose detection, recognizing and tracking multiple human poses within a single frame. This process involves adjusting the current code to manage multiple sets of pose landmarks while ensuring precise identification and tracking of each individual's pose.

Pose Estimation with 3D Depth Information:

- **Objective:** Combine depth sensing technology, like depth cameras, to estimate both 3D pose and 2D landmarks.
- **Overview:** Enhance the script by incorporating depth information alongside RGB data to improve the accuracy of 3D pose estimation. This entails integrating depth sensing devices or depth estimation algorithms to capture depth details, aligning them with RGB images, and integrating them into the pose estimation process for a more comprehensive spatial understanding of human poses.

Real-Time Gesture Recognition:

- **Objective:** Implement gesture recognition to interpret specific hand movements or gestures.
- **Overview:** Enhance the script with gesture recognition functionality by analyzing the sequence of detected hand landmarks over time. Develop and train machine learning models to identify predefined gestures or movements like waving, thumbs-up, or pointing. Integrate the gesture recognition system with the existing pose detection pipeline for real-time interpretation of hand gestures.

Pose-Based Action Recognition:

- **Objective:** Create a system to recognize complex actions based on detected human poses.
- **Overview:** Expand the script to include action recognition capabilities by analyzing the temporal sequences of pose landmarks to identify specific actions or activities in the video. Utilize machine learning techniques like recurrent neural networks (RNNs) or long short-term memory (LSTM) networks to learn patterns of pose sequences corresponding to different actions.

Integration with Augmented Reality (AR) Applications:

- **Objective:** Fuse pose detection with AR technology to develop immersive augmented reality experiences.
- **Overview:** Merge the script with AR frameworks or libraries to superimpose virtual objects or information onto the real-world scene based on detected human poses. Create AR applications that dynamically respond to users' movements and gestures, such as virtual fitness trainers, dance instructors, or interactive gaming experiences. Utilize the detected pose landmarks to accurately position and animate virtual objects within the AR environment, offering users captivating and interactive experiences.

Summary :

The Python script provided combines OpenCV and Mediapipe to identify human poses in a video stream. It starts by importing essential libraries: OpenCV for computer vision, Mediapipe for pose detection, and time for FPS calculation. The script initializes the Mediapipe Pose model for landmark detection and opens the video file "workout.mp4" for processing.

During the main loop, each video frame is read, resized to fit maximum dimensions while preserving the aspect ratio, and converted from the default BGR color space to RGB for Mediapipe processing. The Mediapipe Pose model is then used to detect pose landmarks on the RGB frame.

If pose landmarks are identified, they are visualized on the image using Mediapipe's drawing tools. Furthermore, the script computes the video's FPS and presents it on the frame using OpenCV's text rendering functions.

The processed frame, including the recognized pose landmarks and FPS data, is shown in a window named "Image". The script iterates through each video frame until the user presses the 'q' key to release video capture resources and close all OpenCV windows.

In essence, this script illustrates the seamless fusion of OpenCV and Mediapipe for human pose detection in videos, offering a valuable tool for fitness tracking, gesture recognition, and human-computer interaction. It highlights the adaptability and user-friendliness of these libraries for real-time computer vision applications.